

CLMSynth v0.7.0

Arootin Gharibian

August 30, 2026

Abstract

CLMSynth generates a label $L(x)$ and attaches it to an existing set of cluster IDs $c(x)$ (ground truth or algorithm output), with the relationship between L and c specified by a configuration file and then measured afterwards. A synthetic dataset generated with this program can be used to benchmark and evaluate cluster and label matching under user-definable parameters: clusters with random class assignments and balanced or imbalanced clusters with varying degrees of cluster-label matching. Unlike existing clustering benchmarks that implicitly equate clusters with classes, the generator treats cluster formation and class assignment as independent processes with configurable matching outcomes. Each dataset instance includes ground-truth cluster membership, class labels, and metrics to create reproducible comparative benchmarking of cluster-label matching. A flexible target-metric solver can iterate over possible configurations, respecting user-defined criteria for label assignment, and solve for a CLM function within tolerance of a specific target.

License

CLMSynth Documentation is licensed under Creative Commons Attribution-ShareAlike 4.0 International license. The source code for the program is published under the MIT License at GitHub.

1 Introduction

Existing synthetic datasets for clustering performance measurement assume equivalence between latent cluster structure and assigned class labels. The CLM dataset synthesizer addresses the natural occurrence of classes within latent clusters by treating cluster generation and class assignment as independent, configurable, and target-matched for a specific cluster-label matching. This design supports multiple benchmarks, including non-clusterable data with strong class separability, well-defined clusters with randomly assigned classes, and both balanced and imbalanced clusters. For each configuration, the output is a dataset of labels attached to the cluster membership, along with summary diagnostics characterizing cluster-label matching (CLM) measures. By generating and systematically comparing reproducible CLM scenarios, this resource complements existing clustering benchmarks and supports the evaluation of methods that study the relationships between the clustering structure and class labels.

1.1 Scientific Objectives

Cluster-label matching (CLM) is measured as the feature subset’s effectiveness in predicting ground-truth class labels, the degree to which a feature subset supports known classes or labels regardless of clustering performance. The performance and quality outcomes associated with a selected subset of features, evaluated in terms of its ability to detect structures (clusters) and its alignment with class labels, are independent. Different algorithms and measures can optimize the selection of a given subset of features and assess how it performs in clustering, classification, and cluster-label matching. The objective of this program is to create datasets that give a user granular control over labels and match specific cluster-label matching targets, independent of the clustering, so that an algorithm or measure can reproduce specific CLM targets.

1.2 Implementation

The implementation in Python, together with the pipeline scripts, configuration templates, and working examples, is maintained in the project repository. The source code is the authoritative description of the implementation; this manual is intended for software users and includes the definition of the configuration schema and the troubleshooting reference.

2 Implementation Details

2.1 Label generator

1. **Validate**, structural checks fail fast: `num_classes` and the data's own cluster count K must each lie in $[1, 64]$, a coarse backstop rather than a reliability boundary); `perfect` requires $M = K$; `single` requires `single_match`, $M \geq 2$, $K \geq 2$; `target_metric` is only compatible with `single/custom`, and `scope: pair` additionally requires `type: mcc` and `matching_mode: single`; any requested spatial placement requires per-point coordinates; unknown modes raise errors.
2. **Resolve proportions** p are uniform if `balanced`; otherwise, explicit `proportions` take precedence, with `skew_rule` as a fallback. Converted to integer counts by the largest-remainder rounding.
3. **Matching mode** $\rightarrow$ alignment rules:
 - `random`, a random permutation of the fixed multiset holding each label ℓ_i exactly m_i times, rather than an independent draw from p per point: the realized counts equal m_i , and the permutation consults neither $c(x)$ nor position, so the independence from $c(x)$ holds by construction rather than expectation.
 - `perfect`, bijection $\pi : \text{cluster} \rightarrow \text{label}$ in sorted order; label counts are assigned to the paired cluster sizes; $L(x) = \pi(c(x))$.
 - `single`, one target pair (k^*, ℓ^*) aligned at full recall; all other points are assigned by spillover.
 - `custom`, an `assignment_matrix` of rules, each a `recall_target` fraction of one label's point budget into a chosen set of clusters (enabling surjective, partial, and overlapping).
4. **Allocation & feasibility**, each rule is divided into its clusters (`split_rule`); exceeding the cluster capacity raises an infeasibility error stating the maximum feasible recall. The remaining cluster capacity is filled by `spillover_rule`. (`proportional_to_marginal` preserves the target proportions).
5. **Target metric**: the target metric and the solver vary only the recall level; every setting above is held fixed across probes, so the noise structure changes the solved recall rather than being applied after it.

With `scope: global` (the default), one recall level is solved for the whole partition by a coarse grid scan; with `scope: pair`, the single-pair MCC of the `single_match` pair is inverted in closed form and no search runs. A target above the configuration’s reachable ceiling terminates at the closest achievable value, and a target below its floor terminates against that in the same way. Because R_K is a maximum over matching, and the mean of R_K over all relabeling is exactly 0, the matched value is never negative and its expectation under independent labeling is strictly positive. Therefore, a requested 0.0 is not null.

The engine measures CLM, and a solution that converged internally can still land outside the tolerance, most visible at small N , where the placement of spillovers is a large part of the result.

Currently, three CLM measures are available: `clustering_mcc` is the permutation-invariant `scikit-learn` implementation of Gorodkin’s R_K , which is a one-to-one matching of the cluster-label contingency table that maximizes R_K as the value solved for.

`scope: global`. `clustering_mcc_pair` is the 2×2 Matthews ϕ of a cluster against a label, which does not need matching.

`clustering_ari` is solved for the adjusted Rand index (ARI): The ARI measures the agreement over *pairs* of points rather than through any correspondence between the label and cluster identifiers, and is chance-corrected so that independent partitions score 0. Because it is a function of the cell of the contingency table and the marginal pair counts alone, it does not need an alignment step: permuting the values of the generated label leaves it unchanged, it is symmetric in its two arguments, and it is defined for any M and K . This is what distinguishes it from R_K , which is meaningful only after the Hungarian alignment.

6. **Spatial placement** Configures optionally, within each cluster, *which* points receive each label is chosen by weighted sampling without replacement, with weights derived from normalized distance to the cluster centroid, with `profile: linear/step/exponential`; and `favors: steepness` for exponential or `core/boundary` for other profiles. The contingency table, and therefore the target CLM outcome, is fixed by stages 2–4, so spatial structure can be varied at a constant metric value.

Note on multi-class multi-label measurement: Releases of up to 0.6.9 were matched on the grid-search to maximize precision, and thus reported the R_K of accuracy-optimal matching. The two agree where the rules impose real alignment and differ only near the lower end of the target metric, specifically for unbalanced datasets.

2.2 Configuration file definition

The `clm_label_engine` is the core module for label synthesis. The configuration parameters for the core module are defined as follows:

```
clm_label:
  num_classes: M                                # 1 <= M <= 64; the data's
  own K is capped the                           # same way (CLM-126 / CLM
  -127)
  balance: balanced | unbalanced                # balanced: uniform 1/M,
  proportions ignored
  proportions: [...]                            # unbalanced: used
  directly when given. Must have                # exactly M entries (CLM
  -121) summing to 1 (CLM-106)
  skew_rule: geometric | dominant_minority | dirichlet
  no proportions                                # unbalanced fallback when
  skew_params:
    ratio: 0.5                                  # geometric:  $p_i \sim \text{ratio}^i$ 
    dominant_index: 0                            # dominant_minority
    dominant_share: 0.5                          # dominant_minority
    alpha: 1.0                                  # dirichlet: Dirichlet(
    alpha,...,alpha)
  matching_mode: perfect | single | random | custom
  single_match: {cluster: k*, label: l*}        # required if
  matching_mode: single
  assignment_matrix:                            # required if
  matching_mode: custom
  - {label: l, clusters: [k1, ...], recall_target: r}
  split_rule: proportional_to_size | equal      # how a rule's
  mass splits over its clusters
  spillover_rule: proportional_to_marginal | uniform |
  concentrated
  concentrated_labels: [l, ...]                 # spillover_rule:
  concentrated only. A LIST of                  # integer ids in 0..M-1 (
  CLM-128); a bare number                       # is read as a range, not
  a one-element list                           # (default: the single
  largest label)
  target_metric:                                # optional (single/custom
  only)
  type: mcc | ari
  value: 0.7
  scope: global | pair                          # global (default): one
  recall solved
```

```

whole partition.
for the
requires type: mcc
single.
  tolerance: 0.01
  DELIVERED
the target
  is raised
  max_iter: 40
budget; the pair
and never iterates
  probe_seed: 42
the run seed
competing_noise:
  : structured noise.
cluster's UNCLAIMED
label. Bypasses the
design (CLM-304).
- cluster: k
  label: l_comp
  share: 1.0
's leftover points
  favors: boundary | core | random
centroid_dependence:
  enabled: true | false
  profile: linear | exponential | step
  favors: core | boundary
  steepness: 3.0

```

```

# numerically for the
# pair: exact closed form
# single_match pair;
# and matching_mode:
# both scopes: how far the
# labeling may sit from
# before CLM-309/CLM-310
# global only, bisection
# scope inverts exactly
# optional; defaults to
# optional (single/custom)
# Converts a share of ONE
# points to one competing
# target proportions by
# fraction of that cluster
# exponential only

```

Spatial placement (centroid_dependence or a competing entry _noise that favors the core/boundary) requires vectors of characteristic per-point. The complete catalog of coded errors and warnings is the reference for troubleshooting.

3 Usage

The input variables required for the generator (X and c ; cluster centroids are derived internally from X) are expected to be supplied by external clustered datasets. CLMSynth is designed to work with online and offline datasets, such as the Gagolewski's clustering benchmark suite (`clustbench`) or synthetic datasets generated via MDCGen (`mdcgen`). In addition, synthetic data generated through the offline generator (`fabricated_data`) and direct import through `byoc` (*bring-your-own-clusters*) are possible.

The package installs three entry points: `clmsynth` runs the pipeline with a YAML configuration, `clmsynth-config` renders a configuration from an upstream payload, and `clmsynth-wizard` generates one interactively. A simple demonstrative run is as follows:

```
# Install package with requirements, then:
from clmsynth.clm_label_engine import generate_clm_labels
# An arbitrary clustering data:
rng = np.random.default_rng(0)
sizes = [200, 150, 150]
c = np.repeat([0, 1, 2], sizes)
X = np.vstack([rng.normal(mu, 1.0, size=(n, 2))
               for mu, n in zip([(0, 0), (6, 0), (0, 6)],
                               sizes)])
# Basic configuration:
config = {
    "num_classes": 3,
    "balance": "unbalanced",
    "proportions": [0.5, 0.3, 0.2],
    "matching_mode": "custom",
    "assignment_matrix": [
        {"clusters": [1, 2], "label": 0, "recall_target":
0.9},
    ],
    "split_rule": "proportional_to_size",
    "spillover_rule": "proportional_to_marginal",
    "centroid_dependence": {
        "enabled": True,
        "profile": "exponential",
        "favors": "core",
        "steepness": 4.0,
    },
}
# Generate labels:
L = generate_clm_labels(cluster_labels=c, coords=X, cfg=
config, seed=42)
print(L.value_counts().sort_index().to_dict())
# -> {0: 250, 1: 150, 2: 100}
```

4 Example Outputs

The global R_K /ARI between an M -label and a K -cluster partition has no closed form, which is why `scope: global` is solved numerically. The compact closed-form expression TP^* inverts the *single* 2×2 MCC (what `scope: pair` evaluates), which is why that mode requires `type: mcc` and `matching_mode: single` and reaches its target without a search (see the published article). That is why in table 1 The CLM target of single-match configurations A3 and D1 is feasible. Configurations A4 and B1–B3 share one set of rules (labels sized $[1024, 614, 410]$, one rule per label); B1 and B2 request possible values within their configuration (solver within the tolerance 0.01, while B3, a 3-label partition of a 2-cluster, requests $MCC = 0.99$, which is not possible. The solver returns its closest feasible value, 0.718, which is the maximum attainable for this configuration. C1–C3 differ only in spatial placement.

Cfg	Configuration	Expected	Actual
A1	perfect, $M=K=2$	bijection; $MCC \approx 1$	$MCC = 1.000$, $ARI = 1.000$
A2	random, $M=4$	no cluster info; $ARI \approx 0$, R_K at its null level	$MCC = 0.018$, $ARI = -0.000$
A3	single (cluster 1, $M=4$)	feasible here ($512 < 1024$); partial alignment	$MCC = 0.279$, $ARI = 0.166$
A4	custom, explicit recalls (0.8/0.5/0.3)	feasible; counts $[1024, 614, 410]$	$MCC = 0.527$, $ARI = 0.393$
B1	custom + solve $MCC= 0.6$	reachable here; solver hits it within tol.	solved $MCC = 0.598$ (target 0.6)
B2	custom + solve $ARI= 0.4$	reachable here; solver hits it within tol.	solved $ARI = 0.391$ (target 0.4)
B3	custom + solve $MCC= 0.99$, max_iter 15	unreachable; non-convergence + delivered-value check	solved $MCC = 0.718$
C1	single largest + centroid linear/core	$MCC = A3$ (placement in 4-D)	$MCC = 0.279$, $ARI = 0.166$
C2	single largest + centroid exp/boundary	same contingency as C1	$MCC = 0.279$, $ARI = 0.166$
C3	single largest + centroid step/-core	same contingency as C1	$MCC = 0.279$, $ARI = 0.166$
D1	single (cluster 1, $M=2$)	fills cluster exactly; $MCC \approx 1$	$MCC = 1.000$, $ARI = 1.000$

Table 1: Cluster–label matching outcomes on `g2mg_4_20` ($N = 2048$, 4-D, $K = 2$ clusters of 1024 points, reference labeling `labels0`).

Revision History

Revision	Date	Author(s)	Description
0.1	2026-07-07	AG	Initial release: CLM label engine, [CLM-###] diagnostics, configuration wizard, MCC/ARI evaluation.
0.2	2026-07-15	AG	Internal, unpublished.
0.3	2026-07-31	AG	<code>clustering_mcc_pair</code> added to the API; <code>fabricated_data</code> cluster ids changed from strings to integers.
0.4	2026-07-31	AG	The renderer can now emit <code>target_metric</code> , <code>competing_noise</code> , <code>concentrated_labels</code> , <code>skew_params</code> and <code>steepness</code> ; <code>mdcgen</code> source repaired.
0.5	2026-07-31	AG	Research release: <code>CITATION.cff</code> , [CLM-1xx] configuration errors abort the run.
0.6.2	2026-08-02	AG	First public release. Label-space and pair-scope guards: [CLM-121], [CLM-128], [CLM-129], [CLM-130], [CLM-153] and [CLM-310] reject configurations that previously produced silently incorrect labels; documentation revised throughout
0.6.3	2026-08-08	AG	Correctness release: [CLM-131] validates <code>skew_params</code> ; <code>scope:pair</code> now honors <code>tolerance</code> ; run folders are created atomically; [CLM-104]/[CLM-105] report per dataset instead of aborting a batch.
0.6.4	2026-08-09	AG	Tooling and import hygiene: the target-metric search and the final generation now share one random stream, so a solved value is the value delivered; <code>byoc</code> imports are validated against stated requirements (see the uncoded-rejections section of the troubleshooting reference.

0.6.5	2026-08-10	AG	The test suite becomes part of the repository.
0.6.6	2026-08-11	AG	A patch release
0.6.7	2026-08-12	AG	CLI-Wizard update and improvement
0.6.8	2026-08-20	AG	Patch without release, preparation for CLMSynth-GUI
0.6.9	2026-08-23	AG	Patch release, staging for release 0.7.0
0.7.0b1	2026-08-30	AG	Article-review release: maximize the multiclass <code>R_K</code> rather than accuracy; Brent's method for the recall solver ; <code>byoc_suite.tag_columns</code> carries non-feature columns through an import; support for Python 3.11 dropped.

Licenses for Third-Party Libraries

Python is distributed under the Python Software Foundation License V.2. This project relies on several open-source third-party Python libraries, including NumPy, Pandas, SciPy, scikit-learn, Matplotlib, Seaborn, and PyYAML. Optionally faker and mdcgenpy. These packages are distributed under their respective licenses (BSD 3-Clause, MIT, and Apache 2.0). Refer to the official documentation of each package or the project repository for full third-party license details.

5 References

- Gagolewski, M. (2020). *A framework for benchmarking clustering algorithms*. <https://clustering-benchmarks.gagolewski.com/weave/suite-v1.html>
- Iglesias, F., Zseby, T., Ferreira, D. *et al.* (2019). *MDCGen: A Multidimensional Dataset Generator for Clustering*. Journal of Classification. <https://link.springer.com/article/10.1007/s00357-019-9312-3>
- Jeon, H., Aupetit, M., Shin, D., Cho, A., Park, S., and Seo, J. (2025). *Measuring the Validity of Clustering Validation Datasets*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 47(6), 5045-5058. <https://doi.org/10.1109/TPAMI.2025.3548011>

Configuration Troubleshooting Reference

CLMSynth v0.7.0

August 30, 2026

This reference documents every configuration condition that is either an incompatibility or an ignored setting, as well as the diagnostic message of the CLMSynth label engine. It is intended to help users *plan* a valid configuration and interpret what the engine does with a specific configuration.

How the code works. The identifiers below are a documentation index for this manual; the core label synthesizer engine raises standard Python exceptions with messages banded by type:

- 1xx ValueError** invalid value (**100–110**) or **Incompatible** option (**111–149**) or **InfeasibleAllocationError**, Infeasible outcome (**150–169**).
- 2xx KeyError** a required configuration key is missing (message carries the code).
- 3xx Warning** non-fatal; logged via `logging.warning`, execution continues.

1 Configuration option compatibility at a glance

Option	perfect	single	random	custom
proportions / balance	○	✓	✓	✓
skew_rule				
single_match	–	✓ req	–	–
assignment_matrix	–	–	–	✓ req
split_rule	–	○	–	✓
spillover_rule	○	✓	–	✓
competing_noise	○	✓	×	✓
target_metric	×	✓	×	✓
target_metric.scope: pair	–	✓	–	×
centroid_dependence	○	✓	○	✓

Table 1: Option × matching_mode compatibility: ✓ used / required (**ValueError** or **KeyError** if required but not provided), – not read / not applicable, ○ accepted but ignored or no visible effect (with **Warning**), × not allowed (raises a **ValueError** of type *Incompatible*).

Not every configuration rejection is diagnostic **CLM**. There are conditions that check whether the *data* that were imported by the configuration is a usable clustering at all. They do not carry a code because the solution is outside the scope of the program.

2 Requirements & Restrictions

2.1 Requirements (planning)

The user shall provide, through a configuration file, a required set of configuration settings (Table 2). It is necessary to plan the required configuration before executing the label synthesizer; an independent CLI wizard module (`config_wizard`) provides step-by-step guidance for generation of the configuration file.

Setting	Requires
<code>num_classes</code> (M)	Integer in $[1, 64]$ (≥ 2 under <code>single</code>); the dataset's own cluster count K is capped the same way. A coarse safety backstop (126/127), not a validated reliability boundary, internal testing already finds results unreliable well below this, at $K \gtrsim 15$ or $M \gtrsim 20$.
<code>matching_mode</code>	One of <code>perfect</code> , <code>single</code> , <code>random</code> , <code>custom</code> .
<code>balance:</code> <code>unbalanced</code>	Either explicit proportions <i>or</i> a <code>skew_rule</code> . Explicit proportions need exactly <code>num_classes</code> entries (121) summing to 1 (106). Rules have defaults, so <code>skew_params</code> may be omitted.
<code>skew_params</code>	<code>skew_params</code> for its type: <code>geometric</code> → <code>ratio</code> ; <code>dominant_minority</code> → <code>dominant_index</code> , <code>dominant_share</code> ; <code>dirichlet</code> → <code>alpha</code> .
<code>matching_mode:</code>	
<code>perfect</code>	<code>num_classes</code> == number of clusters ($M = K$); a bijection needs one label per cluster.
<code>single</code>	$M \geq 2$, $K \geq 2$, and <code>single_match</code> : <code>{cluster,label}</code> ; the cluster must exist and the label must be in $0..M-1$. Refer to <code>target_metric.scope</code>
<code>custom</code>	<code>assignment_matrix</code> with ≥ 1 row; each row's <code>label</code> $\in 0..M-1$ and clusters exist; each row needs <code>recall_target</code> <i>unless</i> <code>target_metric</code> is set.
<code>target_metric</code>	<code>matching_mode</code> $\in \{\text{single, custom}\}$; <code>type</code> $\in \{\text{mcc,ari}\}$; <code>value</code> $\in [-1, 1]$.
<code>target_metric.scope</code>	"global" (default) or "pair". pair additionally requires <code>type</code> : <code>mcc</code> and <code>matching_mode</code> : <code>single</code> , and is met in closed form (<code>tolerance</code> / <code>max_iter</code> are not read).
<code>spatial placement</code>	Per-point feature vectors (an (N, d) <code>coords</code> array) whenever <code>centroid_dependence</code> is enabled or a <code>competing_noise</code> entry favors <code>core</code> /boundary; labels-only configs must omit them.
<code>competing_noise</code>	<code>matching_mode</code> $\in \{\text{single, custom}\}$; each entry: <code>cluster</code> exists, <code>label</code> $\in 0..M-1$, <code>share</code> $\in [0, 1]$, <code>favors</code> $\in \{\text{core,boundary,random}\}$. Needs <i>unclaimed</i> (leftover) points to have any effect.
<code>centroid_dependence</code>	<code>profile</code> $\in \{\text{linear,exponential,step}\}$; <code>favors</code> $\in \{\text{core,boundary}\}$, matched exactly and case-sensitively (129); <code>steepness</code> only for <code>exponential</code> .
<code>concentrated_labels</code>	Under <code>spillover_rule</code> : <code>concentrated</code> only: a <i>list</i> of integer label ids, each $\in 0..M-1$ (128). A bare number is not a shorthand for a one-element list. Omit the key to take the default, the single largest label.

Table 2: List of configuration a user is required to plan and provide to the software

2.2 Incompatibilities

Incompatibilities are configurations in which every individual setting is valid but the combination is not: the requested features are mutually exclusive so that no assignment can satisfy both at once.

Feature	Not allowed when	Code	Reason & fix
<code>target_metric</code>	<code>matching_mode = perfect</code>	111	Perfect is a fixed bijection ($MCC=1$); there is no free <code>recall_target</code> to search. Use <code>single/custom</code> , or drop <code>target_metric</code> .
<code>target_metric</code>	<code>matching_mode = random</code>	114	Random ignores clusters entirely; there is no agreement level to dial. Use <code>single/custom</code> .
<code>competing_noise</code>	<code>matching_mode = random</code>	115	There is no aligned label for a competing label to compete with. Use <code>single/custom</code> .
<code>target_metric.scope: pair</code>	<code>type = ari</code>	123	The single-pair MCC inverts in closed form; ARI has no such inverse. Use <code>type: mcc</code> , or <code>scope: global</code> .
<code>target_metric.scope: pair</code>	<code>matching_mode $\neq$ single</code>	124	The target pair is the <code>single_match</code> (cluster, label). Use <code>scope: global</code> for <code>custom</code> .
<code>spatial placement</code>	<code>features</code> missing or empty	125	Centroid distances need complete feature values. Pass an (N, d) feature array, or disable <code>centroid_dependence</code> .
<code>centroid_dependence.favors</code>	not <code>core/boundary</code>	129	Matching is case-sensitive and exact. Any other value, including a capitalization slip (<code>Core</code>) or a typo (<code>boundary</code>), is only read when <code>enabled: true</code> .
<code>target_metric.scope: pair</code>	<code>spillover_rule</code> is <code>uniform</code> ; or <code>concentrated</code> targeting the pair's own label (or left to its default); or a <code>competing_noise</code> entry emitting that label	130	The closed form sizes ℓ^* so that <i>all</i> of it sits inside k^* , and there is no search to correct a miss. Anything that can emit ℓ^* elsewhere silently delivers a different coefficient. Use <code>proportional_to_marginal</code> , whose leftover pool for ℓ^* is empty by construction, or <code>scope: global</code> .

Table 3: Incompatibilities between several configurations, which are reported as 1xx value errors as invalid configuration, but cannot be diagnosed by inspecting any single field.

3 Ignored & Irrelevant Settings (expectations)

If the requirements are met with no incompatibilities in configuration (Section 4.1–4.3), all other mode×option mismatches are not considered as errors and are ignored with a warning (see Section 4.4 for a complete reference).

Setting	Ignored / irrelevant when	Effect
proportions	balance = balanced	Replaced by a uniform $1/M$ split. Warns 301.
proportions, balance, skew_rule	matching_mode = perfect	Label counts are forced to each paired cluster’s size. Warns 302.
skew_rule	balance = unbalanced <i>and</i> proportions given	Explicit proportions take precedence; skew_rule is the fallback only.
per-row recall_target	target_metric is set	Overridden by the single solved global α . Warns 303.
centroid_dependence	matching_mode = perfect	Runs but has no visible effect: each cluster is 100% one label, so placement cannot discriminate.
centroid_dependence	matching_mode = random	Placement never runs: random returns before the placement stage. The <i>coords guard</i> still applies, though, being checked ahead of that early return: enabled: true without feature vectors raises 125 even here.
competing_noise	matching_mode = perfect	No-op: zero unclaimed capacity to convert. Warns 305 per entry.
split_rule	a rule targets a single cluster	Short-circuited (the point mass goes to that one cluster directly).
split_rule, spillover_rule	matching_mode = random	No per-cluster allocation step exists at all in random mode.
single_match	matching_mode $\neq$ single	Not read.
assignment_matrix	matching_mode $\neq$ custom	Not read.
steepness	profile $\neq$ exponential	Unused (only the exponential weight uses it).
concentrated_labels	spillover_rule $\neq$ concentrated	Unused, and not validated in this case; default target is the single largest label. Under concentrated it <i>is</i> validated (128).
tolerance, max_iter	target_metric.scope: pair	Not read; the pair target is met in closed form, not by search.

Table 4: Ignored configurations and their effect.

4 Error and Warnings Reference

4.1 Value Error (invalid configuration) 1xx

Code	Trigger	Reason	Mitigation
101	<code>matching_mode</code> is none of the four modes	Unrecognised mode string	Use <code>perfect</code> , <code>single</code> , <code>random</code> , or <code>custom</code> .
102	<code>perfect</code> with $M \neq K$	A bijection needs exactly one label per cluster	Set <code>num_classes</code> to the true cluster count, or switch to <code>custom</code> .
103	<code>single</code> with $M < 2$ or $K < 2$	Single needs a target label <i>and</i> at least one other, over ≥ 2 clusters	Raise <code>num_classes</code> ; use data with ≥ 2 clusters.
104	<code>single_match.label</code> / a row <code>label</code> not an integer in $0..M-1$	The label id does not exist	Use a value in $0..\text{num_classes}-1$.
105	<code>single_match.cluster</code> / a row <code>cluster</code> not among the data's ids	Typo, or wrong id <i>type</i> (byoc ids are whatever your CSV column holds, and may be strings)	Use an id from the data; match the type (int vs string).
106	<code>explicit_proportions</code> do not sum to 1 (tol. 10^{-6})	Not a probability distribution	Rescale so the entries sum to 1.
107	<code>skew_rule</code> not in <code>{geometric,dominant_minority,dirichlet}</code>	Unrecognised skew rule	Use one of the three, or give <code>proportions</code> directly.
108	<code>split_rule</code> not in <code>{proportional_to_size,equal}</code>	Unrecognised split rule	Use one of the two.
109	<code>spillover_rule</code> not in <code>{proportional_to_marginal,uniform,concentrated}</code>	Unrecognised spillover rule	Use one of the three.
110	<code>profile</code> not in <code>{linear,exponential,step}</code>	Unrecognised placement profile	Use one of the three.
111	<code>target_metric</code> under <code>perfect</code>	No free <code>recall_target</code> to solve for	Use <code>single/custom</code> , or drop <code>target_metric</code> .
112	<code>target_metric.type</code> not <code>mcc/ari</code>	Only these two metrics are solvable	Set <code>type</code> : <code>mcc</code> or <code>ari</code> .
113	<code>target_metric.value</code> outside $[-1, 1]$	MCC/ARI live in $[-1, 1]$	Choose a value in range.
114	<code>target_metric</code> under <code>random</code>	No cluster-label structure to dial	Use <code>single/custom</code> .
115	<code>competing_noise</code> under <code>random</code>	No aligned label to compete with	Use <code>single/custom</code> .
116	<code>competing_noise.favors</code> not <code>core/boundary/random</code>	Unrecognised placement side	Use one of the three.
117	<code>competing_noise.share</code> outside $[0, 1]$	Share is a fraction of leftover points	Choose <code>share</code> in $[0, 1]$.
118	<code>competing_noise.label</code> outside $0..M-1$	The competing label id does not exist	Use a valid label id.
119	<code>competing_noise.cluster</code> not among the data's ids	Typo or wrong id type	Use an existing cluster id.
120	<code>target_metric</code> : every probed $\alpha \in [0, 1]$ is infeasible	The rule clusters cannot hold any recall level (Table 6)	Enlarge the target clusters, lower <code>num_classes</code> , or adjust <code>proportions</code> .

Code	Trigger	Reason	Mitigation
121	<code>proportions</code> length $\neq$ <code>num_classes</code>	A longer list silently enlarged the label space: the surplus entries became label ids that <code>num_classes</code> never declared, written into the dataset through <code>uniform/concentrated</code> spillover. A shorter list left labels unallocated	provided exactly <code>num_classes</code> proportions. Not raised under <code>balance: balanced</code> , where <code>proportions</code> are ignored outright (301).
122	<code>target_metric.scope</code> not <code>pair/global</code>	Unrecognised scope string	Use <code>pair</code> or <code>global</code> .
123	<code>scope: pair</code> with <code>type: ari</code>	Only the single-pair MCC has a closed-form inverse	Use <code>type: mcc</code> , or <code>scope: global</code> .
124	<code>scope: pair</code> outside <code>single</code> mode	The pair is defined by <code>single_match</code>	Use <code>matching_mode: single</code> , or <code>scope: global</code> .
125	placement requested without feature vectors	Centroid distances need (N, d) coordinates	Pass <code>coords</code> , or disable <code>centroid_dependence</code> and <code>spatial_competing_noise</code> .
126	<code>num_classes</code> outside $[1, 64]$	Coarse safety backstop against runaway/typo configs, not a validated reliability limit (see Table 2)	Use a value in $[1, 64]$; results are already unreliable well before this ceiling.
127	the dataset's cluster count $K > 64$	Same backstop, applied to the data rather than the config	Regroup or subsample clusters to $K \leq 64$.
128	<code>concentrated_labels</code> is not a list of integers in $0..M-1$	The list is drawn from directly and written into the label column, so an unchecked value puts a label in the dataset that <code>num_classes</code> never declared. A bare number is read as a <i>range</i> (scattering the remainder over that many labels, the opposite of <code>concentrated</code>); a non-integer is truncated on write	Give a list of existing label ids, e.g. <code>[0]</code> . Checked before the <code>target_metric</code> solver, so a solved score can never be computed over a label that does not exist.
129	<code>centroid_dependence</code> mode <code>.favors</code> not <code>core/boundary</code>	Matched exactly and case-sensitively; any other value, including <code>Core</code> or a typo, was silently treated as <code>boundary</code>	Use lowercase <code>core</code> or <code>boundary</code> . Only read when <code>enabled: true</code> .
130	<code>scope: pair</code> combined with a spillover or noise setting that can place the target label outside its cluster	The closed-form inversion assumes every point of ℓ^* lies in k^* ; <code>uniform</code> spillover, <code>concentrated</code> onto ℓ^* , and <code>competing_noise</code> emitting ℓ^* all break that assumption.	Use <code>spillover_rule: proportional_to_marginal</code> , keep <code>competing_noise</code> off ℓ^* , or use <code>scope: global</code> .
131	<code>skew_params</code> out of range for the chosen <code>skew_rule</code>	Unchecked values mostly did not crash: they produced <i>negative</i> label counts that still summed to N , so largest-remainder rounding was satisfied and the run delivered a labeling nobody requested. The remaining cases raised uncoded <code>ZeroDivisionError/IndexError</code>	Use <code>geometric_ratio</code> ≥ 0 ; <code>dominant_minority_dominant_share</code> $\in [0, 1]$ with integer <code>dominant_index</code> $\in 0..M-1$ and $M \geq 2$; <code>dirichlet_alpha</code> > 0 . Only read when <code>balance</code> is not <code>balanced</code> and no <code>proportions</code> are given; omit <code>skew_params</code> for the defaults.

Table 5: Complete list of invalid configurations.

Invalid configurations in Table 5 are covered by the `config_wizard` CLI utility; thus, to generate valid configurations, the wizard is the recommended tool.

4.2 Infeasible Allocation Error 15x

Code	Trigger	Reason	Mitigation
150	A single rule: $\lceil r \cdot m_\ell \rceil >$ capacity of its clusters	The label’s requested budget exceeds the target clusters’ size	Lower recall_target , add clusters to the rule, or shrink the label via proportions . (Message states the max feasible recall.)
151	Rules jointly claim more than a cluster’s size	Overlapping rules oversubscribe one cluster	Lower the competing recall_targets or spread them across more clusters.
152	competing_noise entries for one cluster claim more than its unclaimed points	Too much structured noise demanded on one cluster	Lower the share value(s).
153	Several assignment_matrix rules name the same label and jointly claim more than its budget	A rule’s recall_target is a fraction of the label’s <i>whole</i> budget, so repeating a label across rules adds those fractions. The overshoot silently broke the requested proportions: proportional_to_marginal spillover clips its pool at zero and cannot pull the count back,	Lower the recalls so they sum to ≤ 1.0 , or write one rule listing all the clusters and let split_rule divide the budget.

Table 6: Infeasible Allocation Error, *Under target_metric*, codes 150/151/152/153 are caught internally and treated as “this α is infeasible” during the search; only 120 (no α works at all) surfaces to the user.

All errors in range 1xx have example configurations documented in the project repository; issues must be addressed after checking these invalid configurations and reported only if previously not covered.

4.3 Key Error (a required key is missing) 2xx

Code	Missing key	Required when & fix
201	num_classes	Always. Add num_classes : M.
202	matching_mode	Always. Add matching_mode : <mode>.
203	skew_rule	When balance : unbalanced and no proportions . Add a skew_rule or explicit proportions .
204	single_match	When matching_mode : single . Add single_match :{cluster,label}.
205	cluster / label inside single_match	Both sub-keys are required. Add the missing one.
206	assignment_matrix	When matching_mode : custom . Add at least one rule.
207	label / clusters inside a rule	Every rule needs both. Add the missing one.
208	recall_target inside a rule	In custom <i>without</i> target_metric . Add recall_target , or add target_metric to solve it globally.
209	value inside target_metric	When target_metric is present. Add value : <target>.

Table 7: Supplementary information for the context of KeyErrors (e.g. [CLM-204]).

4.4 Warnings (non-fatal) 3xx

Code	Trigger	Recommendation
301	<code>balance:</code> <code>balanced</code> and <code>proportions</code> given	Expected. To use your <code>proportions</code> , set <code>balance:</code> <code>unbalanced</code> ; otherwise remove them to silence the warning.
302	<code>perfect</code> ignores <code>proportions/balance/skew_rule</code>	Expected. <code>Perfect</code> fixes counts to cluster sizes; drop those keys to make intent clear.
303	<code>target_metric</code> present with per-row <code>recall_targets</code>	Expected. The global α overrides them; you may delete <code>recall_target</code> from the rows.
304	<code>competing_noise</code> active, or <code>spillover_rule</code> <code>uniform/concentrated</code> moved the delivered counts off their target	Expected: the label marginal is no longer binding, so achieved counts are not held to <code>proportions</code> . One code, two causes; the message names which. For exact counts use <code>spillover_rule:</code> <code>proportional_to_marginal</code> and no <code>competing_noise</code> .
305	A <code>competing_noise</code> entry hits a cluster with no unclaimed points	The cluster is fully claimed (or <code>share</code> rounds to 0): Lower a rule's <code>recall_target</code> , or target a different cluster.
306	<code>target_metric</code> did not converge within tolerance	The target likely exceeds the structural ceiling (e.g. $M \ll K$). The closest feasible value is returned; lower the target, raise <code>max_iter/tolerance</code> , or change <code>proportions/clusters</code> .
307	<code>scope:</code> <code>pair</code> target outside the pair's reachable range	The request is clamped to the nearest reachable value; the message reports the range. Choose a target within it, or change the pair.
308	<code>scope:</code> <code>pair</code> resizes the target label	Expected: the label is sized as a subset of its cluster so the pair MCC meets the target exactly; an explicit proportion for that label is overridden.
309	the CLM achieved <code>target_metric</code> is outside tolerance	The search scores candidates on a fixed probe stream. At the same time, the output is generated on the run's own stream, so a solve that converged internally can still land outside tolerance (most often at small N , where spillover placement dominates the outcome). The achieved value in the message is authoritative, not the requested one. To close the gap: raise <code>tolerance</code> , use a larger dataset, or use <code>scope:</code> <code>pair</code> (closed form, no search).
310	<code>scope:</code> <code>pair</code> : the delivered pair MCC is outside the request	Backstop for the closed form, the counterpart of 309 for the global solver. The combinations known to break the construction are rejected up front (130), so this firing means something else moved ℓ^* out of k^* ; the message reports how many of its points did. The achieved value is authoritative, not the requested one.

Table 8: Warning codes, which indicate scenarios of successfully executed configuration, but with a different outcome than the user would expect.

5 Uncoded Rejections

Not every rejection is a **CLM** diagnostic. The banded codes above describe the *cluster-label matching model*: what a configuration asks the engine to do, and whether that is coherent. The conditions in this section describe something prior, whether the *data* that was handed to the engine is a usable clustering at all.¹

5.1 BYOC input requirements

BYOC is an *import* path, not a generator. The premise is that the user has already clustered a subset of their features with their own algorithm (k-means, fuzzy c-means, or any other) and is bringing the result: the feature subset in which the clustering was computed, plus one column holding each point's cluster. The requirements in the following are derived from that premise rather than from the CLM engine.

Every requirement is checked in a single pass, so a file with several problems reports all of them at once rather than one per attempt. Any failure rejects that dataset; under a multi-dataset configuration, the remaining datasets are unaffected and independently checked.

Requirement	Rejected when	Why
Empty file	No data rows, or no columns	A header alone is not a clustering.
Distinct column names	Any name appears twice	<code>pandas</code> renames the second to <code>name.1</code> , so the column meant is ambiguous, and which of the two originals it holds depends on their order in the file.
No reserved names	A column is named <code>Cohort_Class</code> , <code>GroundTruth_*</code> , <code>Cluster_n</code> or <code>Label_n</code>	The first two are consumed as ground truth and would silently disappear as features; the last two are written into the output, producing a duplicate column in the result.
Declared cluster column	<code>cluster_column</code> missing, not a single name, or absent from the file	The ground-truth partition must be named explicitly; it is never guessed.
Complete cluster column	Any missing value	Every point belongs to a cluster. A blank would become a cluster of its own and change K .
At least two clusters	The cluster column holds one distinct value	A single partition has no structure for a label to agree or disagree with.
Minimum cluster size	Any cluster holds fewer than 3 points	A cluster of one or two points is a stray rather than a cluster.
Numeric features	Any non-cluster column is non-numeric	Every column other than the cluster column is treated as a feature, which define the geometry the clustering was computed from.
Complete features	Any missing value in a feature column	The clustering cannot have been computed from missing values, and centroid distances would evaluate to <code>NaN</code> without complaint.

Table 9: BYOC import requirements. These reject a dataset before any label generation; they are not **CLM** codes, because they concern the data rather than the cluster-label matching program.

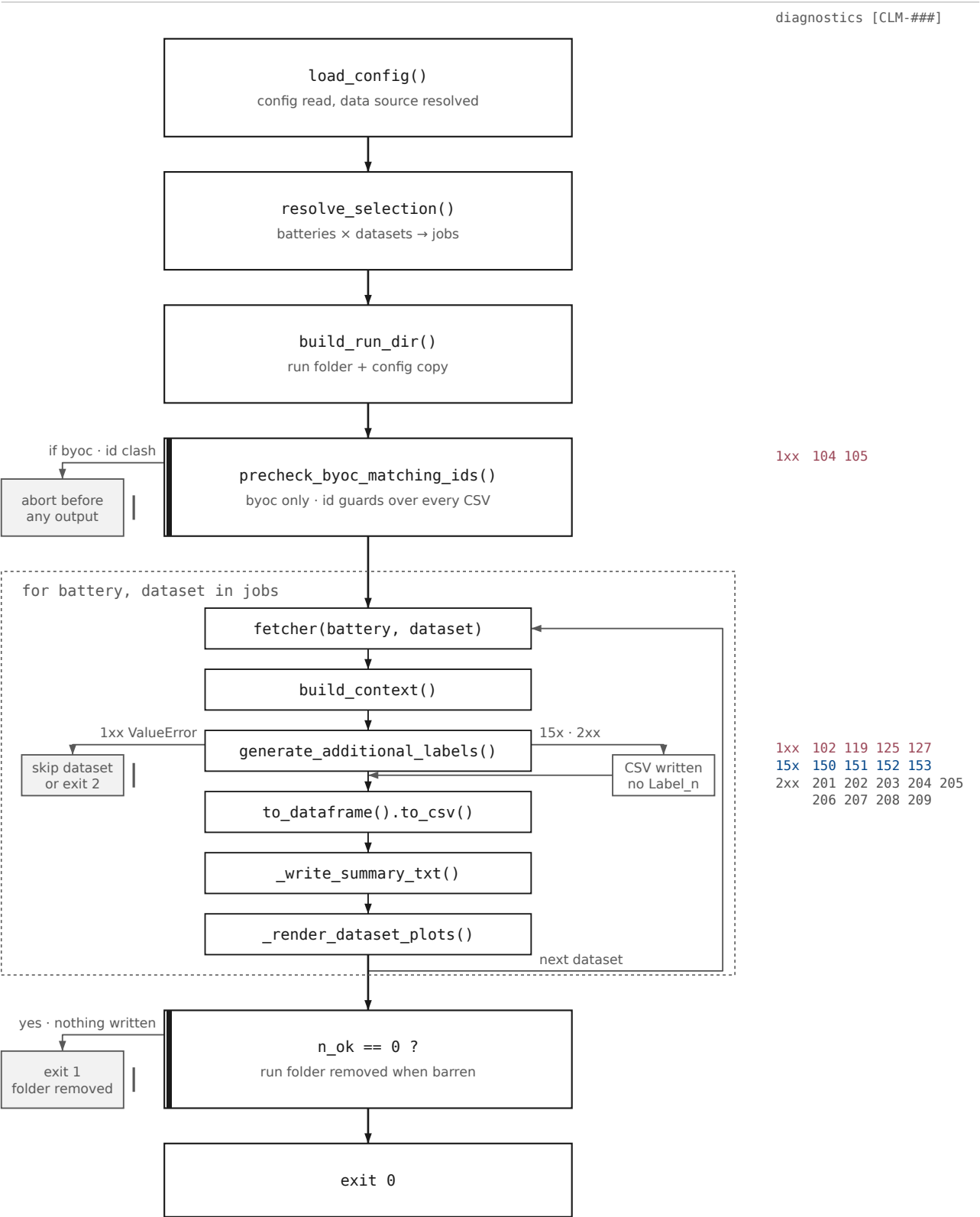
Not restricted. Cluster identifiers may be integers or strings, in any script; if `single_match` or `assignment_matrix` references them, the type must match. The names of the feature columns are carried verbatim to the output CSV and plot axes. Repeated identical rows are kept, since they are additional points rather than a data error, and removing them would change the cluster sizes on which the matching arithmetic is built.

¹This section is expected to grow as more input paths gain checks, and it currently covers `data_source: byoc`.

CLMSynth — run pipeline

clmsynth <config.yaml> → DDMYY_Source_HHMMSS/{csv, png, txt}

codes shown where they are caught



→ arrowhead — continues

— | T-bar — stops here

heavy left rule — branches

left of spine — run or dataset stops

right of spine — returns to the spine

dashed box — one step, expanded

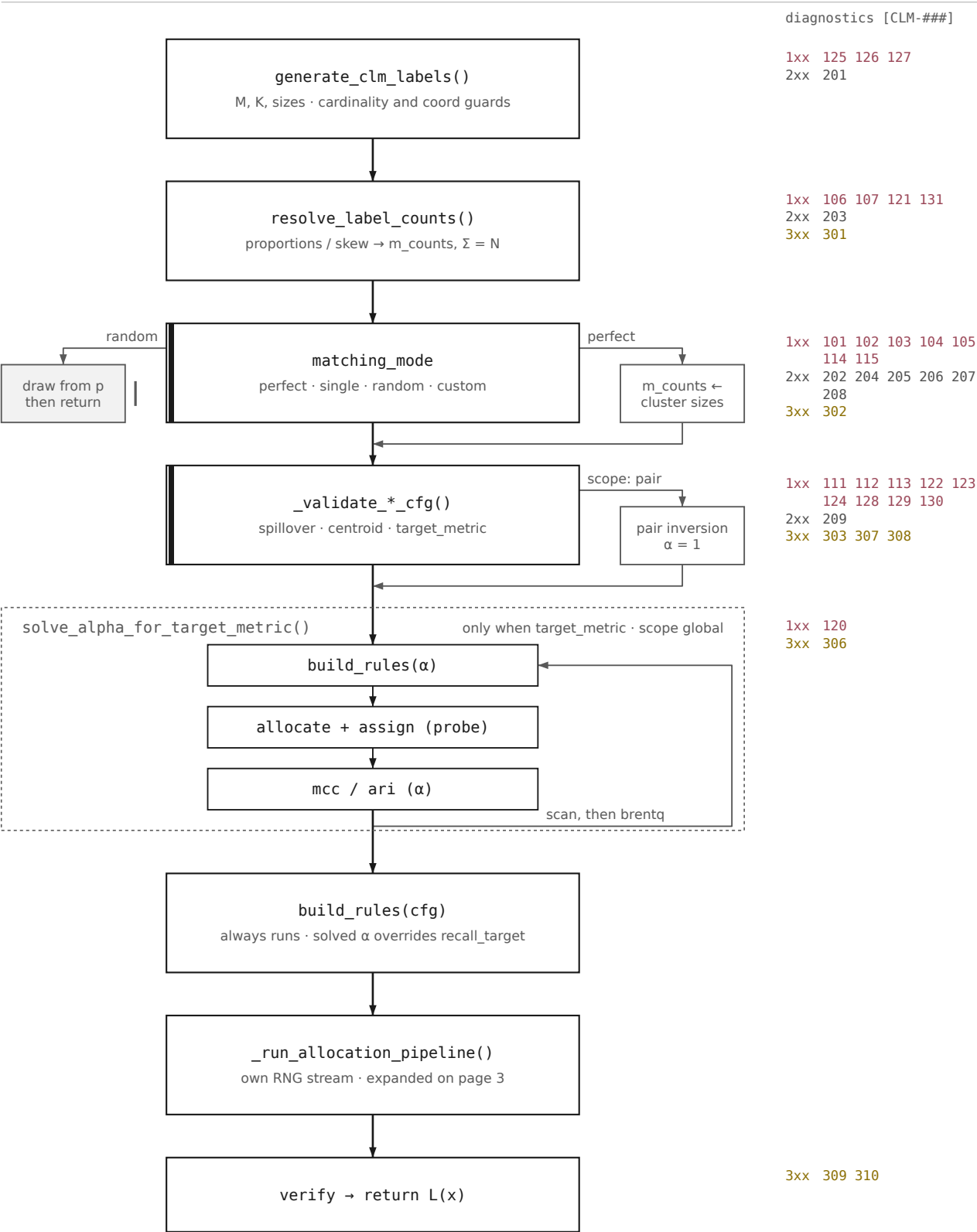
Figure A.1 · outer pipeline (1 of 3) · v0.7.0b1

1xx invalid config · 15x infeasible allocation · 2xx missing key · 3xx warning

CLMSynth — clm_label_engine

$c(x), X, \text{cfg}, \text{seed} \rightarrow L(x) \in \{0 \dots M-1\}$, deterministic under seed

codes shown where they are raised



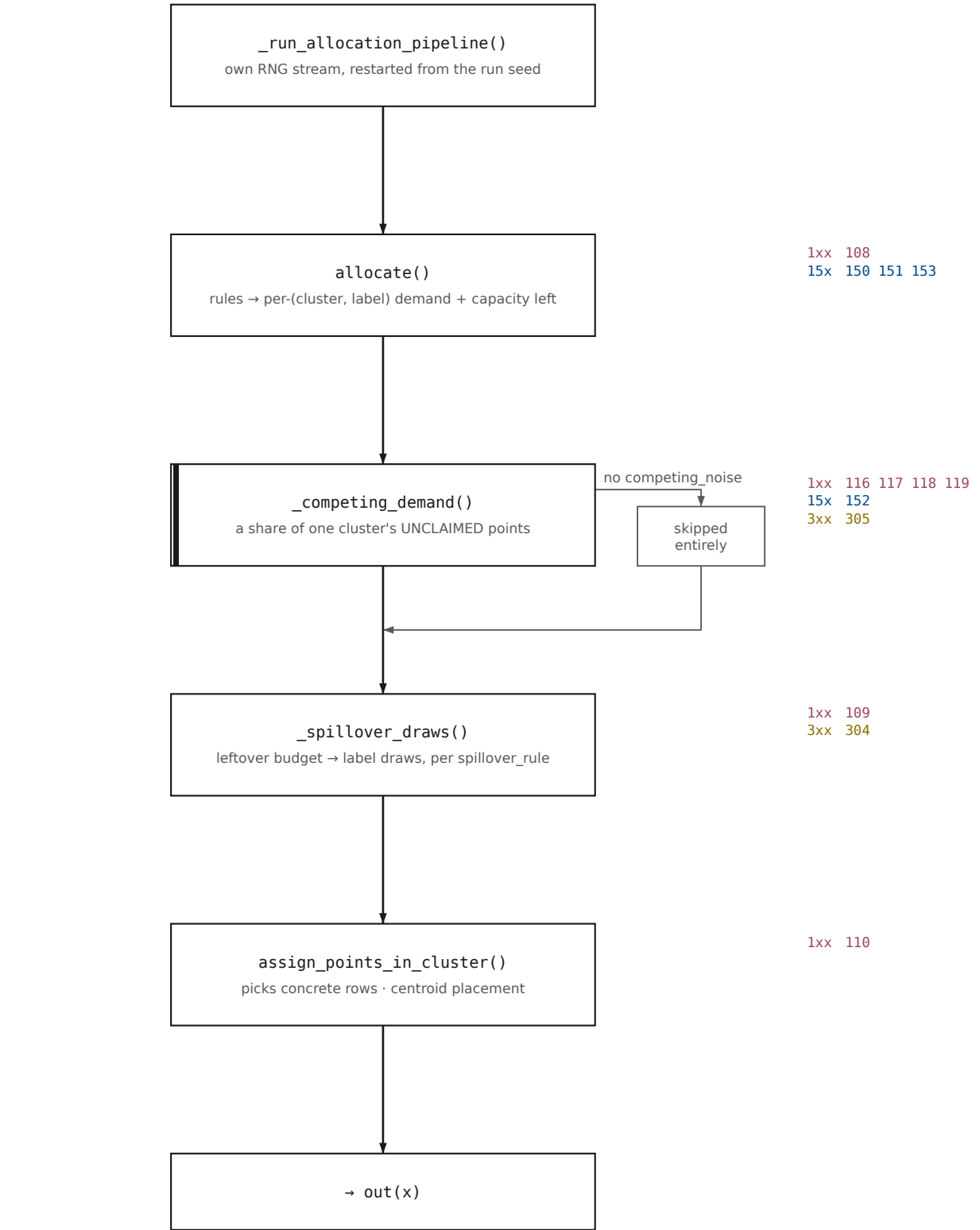
→ arrowhead — continues — | T-bar — stops here **▬** heavy left rule — branches
▬ left of spine — run or dataset stops ▬ right of spine — returns to the spine ▬ dashed box — one step, expanded

CLMSynth — `_run_allocation_pipeline`

rules, m_counts, cluster_sizes → out(x), one label per row

codes shown where they are raised

diagnostics [CLM-###]



→ arrowhead — continues — | T-bar — stops here [] heavy left rule — branches
[] left of spine — run or dataset stops [] right of spine — returns to the spine [] dashed box — one step, expanded

Figure A.3 · allocation and placement (3 of 3) · v0.7.0b1 1xx invalid config · 15x infeasible allocation · 2xx missing key · 3xx warning